

Apuntes Semana 14

Agentes de IA y Modelos de Razonamiento

Julián López Mata
2022207831
Inteligencia Artificial
jullopez@estudiantec.cr

Resumen—Este documento resume los contenidos clave de las sesiones del 26 y 28 de mayo de 2026, enfocados en el diseño y construcción de agentes de inteligencia artificial. Se abordan los componentes fundamentales de un agente, los criterios de selección de modelos, el uso de modelos de razonamiento, el prompting orientado a agentes y la gestión de memoria de corto y largo plazo.

Index Terms—Agentes de IA, Modelos de Razonamiento, Memoria, Context Engineering, Tool Calling, ReAct, Prompting.

I. INFORMACIÓN GENERAL DE EVALUACIONES

Durante la sesión se comunicó que se hace entrega de la última tarea del curso, la cual tendrá un valor equivalente a dos tareas regulares. Su periodo de desarrollo es de dos semanas: la primera parte se entrega en semana 16 y la segunda parte en semana 18. Adicionalmente, el examen final se realizará durante la semana 18.

II. RESUMEN DE LA CLASE ANTERIOR (26 DE MAYO)

II-A. Definición y Componentes de un Agente

Los agentes son modelos que razonan y actúan simultáneamente, mediante la combinación de memoria, herramientas y planificación. Cada agente posee un perfil de personalidad definido por un *system prompt* que le indica cómo debe comportarse, junto con metas y restricciones que delimitan su *scope* de acción. La memoria del agente se divide en dos categorías: memoria de corto plazo, que corresponde a la ventana de contexto del modelo, y memoria de largo plazo, que se mantiene en fuentes externas persistentes.

El proceso de planificación implica descomponer tareas complejas en módulos simples y determinar la secuencia de acciones necesarias para alcanzar un objetivo. Las técnicas más comunes de estructuración incluyen cadenas de pensamiento (CoT), árboles de pensamiento (ToT) e interacción basada en retroalimentación. Una vez planificado, el agente traduce el plan en salidas ejecutables durante la fase de acción, donde las acciones pueden basarse en memorias previas, planes precomputados o herramientas externas tales como RAG, navegación web e integración con APIs.

II-B. Selección de Modelos

Al construir un agente se debe escoger un modelo adecuado en función de su *scope*, considerando factores como accuracy, latencia, costo computacional, privacidad, tamaño de

la ventana de contexto, capacidad de razonamiento y soporte para herramientas. No cualquier modelo funciona para tareas agénticas, y la elección entre modelos propietarios y abiertos depende de los requisitos del proyecto: los propietarios ofrecen fácil integración, infraestructura administrada y buen soporte, pero generan dependencia del proveedor y costos por uso; los abiertos brindan mayor control, privacidad y personalización, a cambio de mayor complejidad operativa. Como nota práctica, para la tarea final se requiere adquirir un API key de un modelo.

III. MODELOS DE RAZONAMIENTO

Los modelos de razonamiento gastan más tokens ejecutando un procesamiento interno con el objetivo de generar respuestas más certeras. Sus respuestas son menos inmediatas, pero pueden ser evaluadas durante el razonamiento para producir una mejor respuesta final al usuario. Resultan especialmente útiles en tareas que requieren matemática avanzada, programación, planificación, resolución de problemas complejos o análisis multi-paso.

Sin embargo, no siempre conviene utilizarlos. En tareas como resúmenes de texto, toma de decisiones sencillas o cálculos básicos, su uso resulta innecesario y costoso. El Cuadro I resume cuándo conviene cada enfoque.

Cuadro I
CUÁNDO UTILIZAR MODELOS CON RAZONAMIENTO COMPLEJO

Sin reasoning complejo	Con reasoning complejo
Clasificación de texto	Resolución de problemas matemáticos
Extracción de información	Análisis de código y debugging
Búsqueda semántica simple	Planificación multi-paso
FAQ bots	Agentes autónomos
Tool routing básico	Coordinación de múltiples herramientas
Traducción y reformateo	Análisis legal o financiero
RAG basado en recuperación directa	Problemas con múltiples restricciones
Conversaciones simples	Tareas complejas de programación

IV. CONSTRUCCIÓN DE AGENTES MODERNOS

IV-A. Modelo Adecuado para Agentes

Un modelo idóneo para construir agentes debe permitir el uso de herramientas, planificación, capacidades de recupera-

ción de memoria, manejo de estados, restricciones y ciclos de ejecución. Por esta razón algunos modelos son más adecuados que otros: deben comprender texto largo, utilizar herramientas externas y coordinar múltiples acciones. Muchos casos de uso requieren además capacidades de *tool calling* y *structured outputs*.

IV-B. Prompting y Context Engineering

En agentes modernos, el prompt no solo solicita una respuesta sino que define el rol del agente, los objetivos de la tarea, las restricciones, las herramientas disponibles, el formato de salida y las reglas de comportamiento. Un buen prompt ayuda a reducir alucinaciones, ambigüedad y respuestas incorrectas. Los componentes principales que estructuran un prompt orientado a agentes se detallan en el Cuadro II.

Cuadro II
COMPONENTES DE UN PROMPT ORIENTADO A AGENTES

Componente	Objetivo
Rol del sistema	Definir comportamiento del agente
Contexto	Agregar información relevante
Instrucciones	Indicar qué debe realizar
Restricciones	Limitar acciones o respuestas
Herramientas	Definir capacidades disponibles
Formato de salida	Controlar estructura de respuesta
Ejemplos	Guiar comportamiento esperado

Estrechamente relacionado con el prompting está el concepto de *context engineering*, que consiste en decidir cuidadosamente qué información entra al contexto, qué información se resume, qué memoria se recupera y qué herramientas se habilitan en cada momento de ejecución.

V. MEMORIA EN AGENTES

V-A. Ventana de Contexto vs. Memoria

Sin memoria, los agentes pueden perder coherencia, repetir preguntas, olvidar restricciones y reiniciar tareas constantemente. Es importante distinguir entre la ventana de contexto y la memoria propiamente dicha: la primera es información temporal procesada directamente por el modelo, limitada por la cantidad de tokens y existente solo durante la ejecución actual; la segunda es información persistente entre interacciones, administrada externamente por el agente, capaz de crecer dinámicamente y almacenarse a largo plazo. Esta distinción se materializa en dos tipos operativos de memoria, comparados en el Cuadro III.

Cuadro III
COMPARACIÓN ENTRE WORKING MEMORY Y LONG-TERM MEMORY

Working memory	Long-term memory
Temporal	Persistente
Activa durante la tarea	Disponible entre sesiones
Información reciente	Información histórica
Cambia constantemente	Más estable
Normalmente limitada por tokens	Puede crecer dinámicamente
Alta relevancia inmediata	Relevancia contextual

Idea clave

La **working memory** ayuda al agente a resolver la tarea actual. La **long-term memory** ayuda al agente a mantener continuidad en el tiempo.

V-B. Working Memory y Long-term Memory

La working memory contiene la información activa durante la tarea actual: conversación reciente, resultados recientes de herramientas, objetivos temporales y restricciones activas. Esta memoria cambia constantemente durante la ejecución y usualmente reside dentro de la ventana de contexto o en un caché interno. La long-term memory, en cambio, persiste información entre sesiones e incluye elementos como preferencias de usuario, historial resumido, proyectos anteriores, patrones de comportamiento y conocimiento persistente; se almacena en bases relacionales, bases vectoriales y sistemas como Redis.

V-C. Compresión y Políticas de Olvido

En tareas largas la memoria crece rápidamente, por lo que los agentes modernos utilizan técnicas de *summarization*, *compression* y *context pruning* para mantenerla manejable. Además, no toda la información debe almacenarse permanentemente: se definen políticas de olvido mediante expiración de memoria, eliminación automática y límites de almacenamiento, motivadas por razones de privacidad, seguridad, relevancia temporal y reducción de costos.

VI. PATRONES ARQUITECTÓNICOS

Durante la sesión se discutieron dos patrones arquitectónicos relevantes para la construcción de agentes. El primero, **ReAct (Reason + Act)**, plantea un ciclo en el cual el agente intercala pasos de razonamiento (*Thought*) con acciones sobre herramientas (*Act*) y observaciones del entorno (*Obs*) hasta alcanzar una respuesta final. Este patrón permite al modelo corregir su trayecto a partir de la información que va recibiendo del exterior, en lugar de comprometerse a un único plan inicial. La Fig. 1 ilustra un ejemplo concreto de este flujo.

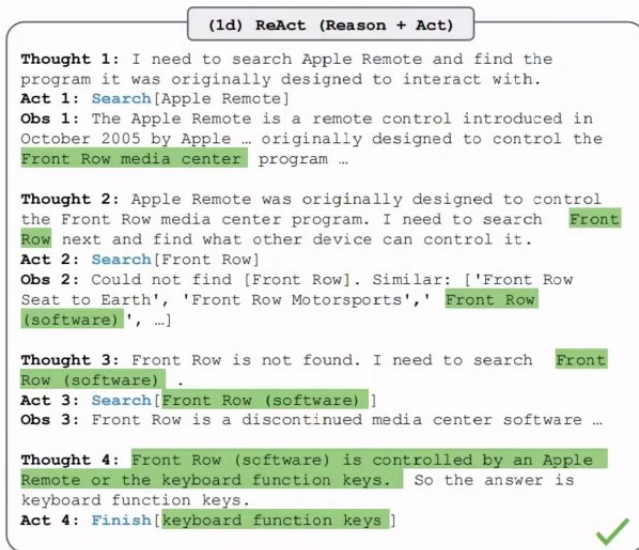


Figura 1. Ejemplo del patrón ReAct intercalando *Thought*, *Act* y *Obs*.

El segundo patrón, **LOMAR**, organiza el ciclo de vida del agente en cinco bloques fundamentales: el LLM como núcleo del sistema, el objetivo (Objective) que orienta la tarea, la memoria (Memory) que mantiene contexto y conocimiento, la acción (Action) que materializa las decisiones, y la re-evaluación (Rethink) que cierra el ciclo permitiendo revisar y ajustar el comportamiento. La estructura completa se muestra en la Fig. 2.

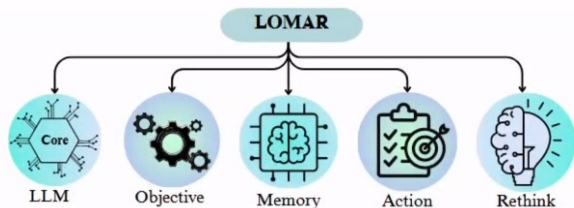


Figura 2. Arquitectura LOMAR: LLM, Objective, Memory, Action y Rethink.

VII. RECOMENDACIONES Y LÍNEAS DE INVESTIGACIÓN

Como recomendaciones generales se enfatizó escalar responsablemente al tomar decisiones sobre cambios en la configuración del agente, así como mantener un buen manejo de datos y evitar sobreexponer información sensible. Para profundizar en el área se sugieren cuatro líneas de investigación: los procesos de comunicación en esquemas multi-agente, la optimización de recursos en sistemas agénticos, el manejo de plataformas web y sus verificaciones anti-bot.

VIII. EXPLICACIÓN DE LA TAREA FINAL

La tarea final consiste en diseñar e implementar un sistema multiagente capaz de responder preguntas a partir de los

apuntes generados por los estudiantes del curso, combinando RAG, orquestación entre agentes, memoria conversacional, uso controlado de herramientas externas y observabilidad.

VIII-A. Componentes del Sistema

El sistema debe integrar una base documental con los apuntes del curso acompañados de metadatos relevantes (autor, fecha, tema, semana, sección), procesados mediante limpieza textual, segmentación y vectorización con un modelo de embeddings de calidad. Sobre esta base operan cinco agentes especializados, descritos en el Cuadro IV.

Cuadro IV
AGENTES DEL SISTEMA MULTIAGENTE

Agente	Función
Orquestador	Coordina el flujo y decide qué agente o herramienta usar (modelo robusto, ej. Sonnet)
RAG	Consulta la base vectorial de apuntes y genera respuestas fundamentadas
Web Search	Búsquedas en internet bajo solicitud explícita o justificada
Resumidor	Resume conversaciones y resultados (modelo ligero)
Transaccional	Consulta una base de datos ficticia vía MCP Server

VIII-B. MCP Server y Reglas de Seguridad

Cada grupo construye un MCP Server que expone herramientas controladas sobre una base de datos transaccional ficticia (clientes, cuentas, transacciones, casos de revisión). Las reglas mínimas incluyen: toda llamada debe incluir justificación, no se permite SQL libre, no se exponen números de cuenta completos, no se permiten modificaciones a transacciones existentes, las consultas sensibles deben anonimizarse y las búsquedas históricas deben acotarse por rango de fechas.

VIII-C. Memoria y Observabilidad

Se debe implementar memoria temporal por sesión (coherencia conversacional) y memoria histórica persistente almacenada en base vectorial. La observabilidad se monta con *Langfuse* y debe registrar entradas del usuario, herramientas invocadas, fragmentos recuperados, llamadas al MCP, latencia y errores. Se recomienda configurar la observabilidad desde el inicio del desarrollo.

VIII-D. Interfaz y Evaluación

El sistema se expone mediante una interfaz web (ej. Streamlit). La evaluación experimental incluye 10 preguntas factuales sobre apuntes, 5 de comparación, 5 conversacionales de seguimiento y 5 fuera de alcance. Adicionalmente se realiza una demostración presencial en semana 18, donde el equipo levanta el ambiente localmente.

VIII-E. Entregables y Fechas

La entrega se divide en dos partes: un avance en semana 16 (mínimo agentes RAG y Web Search funcionando) y la entrega final en semana 18. Los entregables incluyen código fuente (agentes, MCP Server, scripts de poblamiento), archivos SQL, capturas de Langfuse y un informe en LaTeX con introducción, arquitectura, diseño de agentes, RAG, MCP, reglas de seguridad, memoria, observabilidad, evaluación y conclusiones.